VERIFICATION GUIDE FOR STABILIZER-STATE IDENTIFICATION

This repository contains two independent Lean 4 projects accompanying papers by Masahito Hayashi and Yimin Lu. This guide describes the accepted formal scope and how to verify an exact release. It does not certify a future commit.

1. Paper correspondence

arXiv:2609.13923. *Partial Stabilizer Learning under Fixed Commuting Constraints and the Absence of a Copy-Rate Discount* [1]. The root project uses the namespace `StabilizerLearning`. Its main endpoint is

`StabilizerLearning.fixedW_noCopyRateDiscount`.

Its exact type is `StabilizerLearning.FixedWNoCopyRateDiscount`. The complete original fixed-W target includes the common-J converse, four uniform limits to zero, four uniform limits to one under the original condition $k_n - n \to +\infty$, and the exact copy-rate identities. The formalization retains every prime local dimension, the actual states and POVMs, and the original order of worst-case optimization. This is closure of the original main theorem, not a claim that every auxiliary statement in the manuscript is formalized. The itemized coverage and remaining interfaces are in `docs/theorem-map.md`.

arXiv:2610.00266. *Exact Critical Curve for Uniform Stabilizer-State Identification* [2]. The independent project under `proofs/arxiv-2610.00266/` uses the namespace `ExactCriticalCurve`. Its three main endpoints are `finiteSpectrum`, `criticalCurve` and `criticalCurveTails`, all in that namespace. They cover the actual finite Gram spectrum and optimal PGM for $n, k \geq 1$, the literal limiting critical curve for every fixed integer offset c, the exact first-moment identity, and both coefficient-one tails. Additional results cover actual stochastic-group orders, ordinary/nonzero/zero spectral-mass limits, and the exceptional-amplitude inequality. The nested `docs/theorem-map.md` gives the precise quantifiers and endpoint conventions.

The package does not certify the displayed numerical evaluation at $c = 0$, a joint limit with a growing offset, full spectral-measure weak convergence, or a complete exceptional shortening equality merely from the amplitude bound. The formal convergence proof uses sufficient summable bounds and does not certify the paper's exact displayed auxiliary majorant.

2. Meaning of verification

The strict gates check compilation, recursive axiom dependencies, independently expanded theorem types, and required original proof bodies. Only `propext`, `Classical.choice` and `Quot.sound` are permitted by the recursive audits; placeholders or new mathematical axioms do not pass.

These checks certify formal declarations relative to the pinned Lean kernel and libraries. They do not independently decide whether every formal definition faithfully expresses the intended paper semantics. Recorded semantic reviews are AI reviews. Independent human mathematical review remains outstanding.

Date: 15 October 2026.

3. Reproducing an immutable release

Supported pinned compiler platforms are Linux x86_64 and macOS ARM64. Install Git, Python 3.10 or newer, curl, tar and zstd. Keep the two projects' dependencies, compiler directories and build outputs separate. The second complete audit may take several hours.

Obtain the repository from the release repository, then check out the exact commit to be tested. Run all commands below from its root. Record `git rev-parse HEAD` and `git rev-parse HEAD^{tree}` and confirm that `git status -porcelain` is empty.

First paper.

```
python3 scripts/bootstrap.py
./scripts/lake.sh exe cache get $(cat scripts/cache-imports.txt)
python3 scripts/check.py --require-main-proof
```

Also kernel-check the independently expanded original Main statement:

```
expected=docs/validation/2026-10-11-direct-integration
./scripts/lake.sh env lean \
  "$expected/independent-expected/IndependentAcceptance.lean"
```

Require successful command exits and the independent acceptance marker. The strict gate writes its source hashes and logs under `artifacts/check/`.

Second paper.

```
python3 scripts/verify_arxiv_2610.py \
  --dest /absolute/new/reproduction/path
```

The destination must not already exist. The wrapper reconstructs the immutable checked-out commit, obtains its pinned public dependencies, begins with no project proof artifacts, and runs the complete strict release gate. Official Mathlib dependency caches are used by default. The option `-dependency-mode source` instead compiles the required dependency modules from their pinned source. Neither mode rebuilds the Lean compiler from C++ sources.

4. Pins, provenance and release acceptance

Both projects pin Lean 4.24.0 and Mathlib at the same commits:

```
Lean:  797c613eb9b6d4ec95db23e3e00af9ac6657f24b
Mathlib:  f897ebcf72cd16f89ab4577d0c826cd14afaafc7
```

Each project retains its own complete transitive lock. Historical evidence retains its original frozen inputs and execution paths. Those paths are records of earlier checks, not public reproduction dependencies.

GitHub Actions runs both strict gates on pushes and pull requests. For the final `main` commit, require both jobs to succeed and inspect the artifacts for the exact commit/tree, unchanged input hashes, all command exits and complete logs. A prior successful run is historical evidence, not acceptance of later edits. The release preparation preserves the mathematical source, paper snapshots, version pins and verification tooling.

5. Citation for manuscript revisions

Cite the versioned v1.0.0 release, using `CITATION.bib`, together with the relevant paper or papers. The release record identifies the tested commit and tree; the evidence bundle retains the new command logs, axiom audits and type/body checks for both projects. A source-file checksum manifest binds the cited snapshot. The moving `main` branch is not a substitute for the version tag. The documented scope boundaries continue to apply.

References

[1] M. Hayashi and Y. Lu, *Partial Stabilizer Learning under Fixed Commuting Constraints and the Absence of a Copy-Rate Discount*, arXiv:2609.13923v1 (2026).

[2] M. Hayashi and Y. Lu, *Exact Critical Curve for Uniform Stabilizer-State Identification*, arXiv:2610.00266v1 (2026).